

INTRODUCCIÓN A LOS SISTEMAS OPERATIVOS

Resumen teórico - Primer parcial de promoción

Materia	Introducción a los Sistemas Operativos (ISO)
Alcance	Tema 1: Introducción · Tema 2: CPU y Procesos · Tema 3: Administración de Memoria
Criterio	Organizado según la guía de estudio y construido sólo con los PDFs aportados. Cuando la guía pide algo que los PDFs no desarrollan, se marca como hueco en lugar de completarlo con teoría externa.

Objetivo: tener un único material de estudio para el teórico de promoción del primer parcial, conservando la terminología y el enfoque de la cátedra.

Estructura del parcial

- Parte práctica: Práctica 1 + Práctica 2.
- Promoción teórica: evaluación teórica correspondiente al primer parcial. La materia tiene dos evaluaciones teóricas de promoción, una por parcial.

Índice de estudio

Sección	Contenido
1	Tema 1 - Introducción
1.1	Qué es un Sistema Operativo y objetivos
1.2	Apoyo del hardware
1.3	Componentes del SO
1.4	System Calls y arquitectura
2	Tema 2 - Administración de CPU y Procesos
2.1	Programa, proceso, PCB y espacio de direcciones
2.2	Scheduling y objetivos
2.3	Schedulers, loader, dispatcher y context switch
2.4	CPU-bound / I/O-bound
2.5	Estados y transiciones
2.6	Creación y terminación: fork, execve, wait, exit
3	Tema 3 - Administración de Memoria
3.1	MMU, direcciones y espacio de direcciones
3.2	Particiones fijas y dinámicas
3.3	Segmentación
3.4	Paginación
3.5	Segmentación paginada
3.6	Huecos explícitos entre guía y PDFs
4	Repaso final y checklist

TEMA 1 - INTRODUCCIÓN

1. ¿Qué es un Sistema Operativo?

Un Sistema Operativo (SO) es un software que actúa como intermediario entre el usuario de una computadora y su hardware. Al ser software, necesita procesador y memoria para ejecutarse.

- Gestiona el hardware.
- Controla la ejecución de los procesos.
- Funciona como interfaz entre las aplicaciones y el hardware.
- Actúa como intermediario entre el usuario y el hardware.

Dos perspectivas para analizar el SO

Perspectiva	Idea central
Desde el usuario	El SO abstrae la arquitectura, oculta el hardware y presenta abstracciones más simples de manejar. Los programas de aplicación son los clientes del SO.
Desde el sistema	El SO actúa como administrador de recursos y administra los recursos de hardware entre uno o más procesos.

Objetivos del Sistema Operativo

Objetivo	Significado
Comodidad	Hacer más fácil el uso del hardware.
Eficiencia	Hacer un uso eficiente de los recursos del sistema.
Evolución	Permitir incorporar nuevas funciones sin interferir con las anteriores.

Respuesta corta de parcial: Un Sistema Operativo es un software que actúa como intermediario entre el usuario y el hardware. Gestiona los recursos del sistema, controla la ejecución de los procesos y proporciona una interfaz entre las aplicaciones y el hardware. Sus objetivos son comodidad, eficiencia y evolución.

2. Apoyo del hardware requerido por el SO

El SO necesita mecanismos de hardware para poder proteger los recursos y mantener el control del sistema. Los PDFs destacan tres mecanismos fundamentales: modos de ejecución, interrupción de clock y protección de memoria.

Modos de ejecución

El hardware distingue modos de ejecución y limita qué instrucciones pueden ejecutarse en cada uno. Un bit de la CPU indica el modo actual.

Modo	Características
Kernel / Supervisor	Permite ejecutar instrucciones privilegiadas y acceder a estructuras internas del kernel. El kernel del SO se ejecuta en este modo.
Usuario	Permite sólo un subconjunto de instrucciones. El proceso puede acceder únicamente a su propio espacio de direcciones. Los programas de usuario se ejecutan en este modo.

Cuando ocurre un trap o una interrupción, el bit de modo pasa a Kernel. El proceso de usuario no realiza explícitamente ese cambio.

Interrupción de clock

Se utiliza para evitar que un proceso se apropie indefinidamente de la CPU. El hardware puede interrumpir la ejecución y devolver el control al Sistema Operativo.

Protección de memoria

Se deben delimitar las direcciones de memoria a las que puede acceder cada proceso. El ejemplo de los PDFs utiliza Registro Base y Registro Límite. El kernel carga esos registros mediante instrucciones privilegiadas, por lo que esa operación sólo puede realizarse en modo Kernel.

Protección de E/S y CPU

Las instrucciones de E/S son privilegiadas y deben ejecutarse en modo Kernel. Los procesos de usuario solicitan E/S mediante servicios del SO. La protección de CPU se apoya en la interrupción de clock.

Idea clave: no alcanza con que el kernel defina una regla: el hardware debe poder impedir que un proceso de usuario ejecute instrucciones privilegiadas, acceda a memoria ajena o monopolice la CPU.

3. Componentes de un Sistema Operativo

Componente	Función
Kernel	Núcleo del SO. Administra los recursos y brinda servicios esenciales.
Shell / GUI / CUI / CLI	Interfaz mediante la cual se interactúa con el sistema.
Herramientas	Editores, compiladores, librerías y otras utilidades.

Kernel

El kernel permanece en memoria principal, administra los recursos de hardware e implementa servicios esenciales. En los PDFs aparecen servicios vinculados con memoria, CPU, procesos, comunicación/concurrencia y E/S.

Kernel != Sistema Operativo completo. El kernel es un componente del Sistema Operativo.

4. System Calls

Una System Call o llamada al sistema es el mecanismo mediante el cual un programa de usuario accede a servicios del Sistema Operativo. La ejecución del servicio ocurre en modo Kernel/Supervisor.

Secuencia conceptual

Programa en modo usuario -> solicita System Call -> trap/excepción -> modo Kernel -> el kernel identifica y ejecuta el servicio -> retorno al proceso.

Para iniciar una llamada se indica el número de system call y sus parámetros. Los parámetros pueden pasarse mediante registros, bloques/tablas en memoria o pila.

Categorías de System Calls

- Control de procesos.
- Manejo de archivos.
- Manejo de dispositivos.
- Mantenimiento de información del sistema.
- Comunicaciones.

Implementación según arquitectura

Arquitectura	write	exit	Nro. syscall	Parámetros	Activación
x86 32 bits	4	1	EAX	EBX, ECX, EDX, ESI, EDI	int 80h
x86 64 bits	1	60	EAX	RDI, RSI, RDX, R10, R8, R9	syscall

Conclusión del anexo: según la arquitectura cambian el número de System Call, la forma de pasar parámetros y la instrucción utilizada para activarla.

TEMA 2 - ADMINISTRACIÓN DE CPU Y PROCESOS

1. Programa vs. Proceso

Un proceso es un programa en ejecución. Para la materia, proceso, tarea y job se consideran sinónimos.

Programa	Proceso
Es estático.	Es dinámico.
No tiene Program Counter.	Tiene Program Counter.
Existe desde que se edita hasta que se borra.	Su ciclo de vida va desde que se solicita su ejecución hasta que termina.

Para poder ejecutarse, un proceso incluye como mínimo Text/Código + Datos + Stack(s). El stack almacena, entre otras cosas, parámetros, variables temporales/locales y direcciones de retorno.

2. Información que mantiene el Kernel - PCB

Cada proceso posee una estructura asociada llamada PCB (Process Control Block). Existe una PCB por proceso: es lo primero que se crea cuando se crea el proceso y lo último que se elimina cuando termina.

- PID y PPID.
- Registros de CPU.
- Program Counter.
- Estado.
- Prioridad.
- Tiempo de CPU consumido.
- Ubicación/representación en memoria.
- Información de accounting.
- Información de E/S.

Espacio de direcciones != PCB. El espacio de direcciones contiene las direcciones ocupadas por stack, text y datos. La PCB y las estructuras asociadas no forman parte del espacio de direcciones del proceso.

3. Objetivos de los algoritmos de Scheduling

Los algoritmos de planificación pueden perseguir metas distintas según el ambiente.

Meta general	Significado
Equidad	Otorgar una parte justa de CPU a cada proceso.
Balance	Mantener ocupadas todas las partes del sistema.

Procesos Batch

No existen usuarios esperando una respuesta en una terminal. Pueden utilizarse algoritmos no apropiativos. Sus metas incluyen maximizar rendimiento, minimizar tiempo de retorno y mantener la CPU ocupada. Los ejemplos indicados por el material son FCFS y SJF.

Procesos Interactivos

Necesitan algoritmos apropiativos para evitar que un proceso monopolice la CPU. Sus metas son tiempo de respuesta y proporcionalidad. Ejemplos: Round Robin, Prioridades, Colas Multinivel y SRTF.

4. Apropiativo vs. No Apropiativo

Tipo	Idea
Apropiativo / Preemptive	Un proceso que se está ejecutando puede ser expulsado de la CPU contra su voluntad.
No apropiativo / Non-Preemptive	El proceso conserva la CPU hasta que él mismo la abandona: termina con exit, se bloquea voluntariamente con wait/sleep, o solicita una E/S bloqueante como read/write. No se toman decisiones de planificación por interrupciones de reloj.

5. Módulos de planificación

Son módulos de software del Kernel vinculados con la planificación. Se ejecutan ante eventos como creación/terminación de procesos, eventos de sincronización o E/S y finalización de un lapso de tiempo.

Módulo	Función
Long Term Scheduler	Controla el grado de multiprogramación, es decir, la cantidad de procesos en memoria. Puede no existir como módulo separado y su tarea puede ser absorbida por el Short Term Scheduler.
Loader	Carga en memoria el proceso elegido por el Long Term Scheduler.
Short Term Scheduler	Determina cuál de los procesos Ready se ejecutará a continuación. Aquí interviene el algoritmo de planificación.
Dispatcher	Pone efectivamente a ejecutar el proceso seleccionado: realiza cambio de contexto, cambio de modo y salto a la instrucción correspondiente.
Medium Term Scheduler	Relacionado con swapping. Puede reducir el grado de multiprogramación sacando procesos de memoria (swap out) y luego reingresándolos (swap in).

6. Context Switch

El contexto de un proceso contiene la información que necesita el SO para administrarlo y la CPU para ejecutarlo correctamente: registros, PC, prioridad, E/S pendiente, etc.

Un Context Switch ocurre cuando la CPU cambia de un proceso a otro: guardar contexto del proceso saliente -> cargar contexto del proceso entrante -> continuar desde donde corresponda.

El cambio de contexto representa tiempo no productivo de CPU y su duración depende del soporte de hardware.

7. CPU-Bound e I/O-Bound

Los procesos alternan ráfagas de CPU y de E/S.

Tipo	Comportamiento
CPU-bound	Pasa la mayor parte del tiempo utilizando la CPU.
I/O-bound	Pasa la mayor parte del tiempo esperando E/S.

Como la CPU es mucho más rápida que los dispositivos de E/S, el material plantea la necesidad de atender rápidamente procesos I/O-bound para mantener los dispositivos ocupados y aprovechar la CPU con procesos CPU-bound.

8. Estados de un proceso y transiciones

Modelo mínimo:

NEW -> READY -> RUNNING -> WAITING -> READY

Estado	Descripción
NEW	El proceso acaba de ser creado. Se crean sus estructuras asociadas y queda esperando ser cargado en memoria.
READY	Está en memoria y sólo necesita CPU. Permanece en la Ready Queue.
RUNNING	Fue seleccionado y tiene asignada la CPU.
WAITING	Espera que ocurra un evento, por ejemplo finalización de una E/S o llegada de una señal. Continúa en memoria, pero no tiene CPU.

Transiciones principales

- NEW -> READY: Long Term Scheduler elige el proceso y el Loader lo carga.
- READY -> RUNNING: Short Term Scheduler lo selecciona y el Dispatcher le asigna CPU.
- RUNNING -> WAITING: el proceso queda esperando un evento.
- WAITING -> READY: finaliza la espera.
- RUNNING -> READY: puede ocurrir por expulsión, por ejemplo al terminar el quantum en un algoritmo apropiativo.
- RUNNING -> TERMINATED: el proceso finaliza.

Modelo UNIX ampliado

Los PDFs muestran además estados relacionados con swapping y ejecución kernel/user: Ready/Waiting en memoria o swapeado, Created, Preempted y Zombie. El estado Zombie aparece después de exit: el proceso ya no continúa ejecutándose, pero se conserva información como datos de uso y código resultante del exit.

9. ¿Qué schedulers realizan las transiciones?

Transición / acción	Módulo indicado por el material
NEW -> READY	Long Term Scheduler + Loader
READY -> RUNNING	Short Term Scheduler + Dispatcher
Swap out / Swap in	Medium Term Scheduler
RUNNING -> WAITING / WAITING -> READY	Los PDFs describen los eventos y transiciones, pero no asignan explícitamente esta

10. Creación de procesos

Un proceso es creado por otro proceso. Un proceso padre puede tener uno o más hijos, formando un árbol de procesos.

Al crear un proceso se realizan actividades como:

- Crear la PCB.
- Asignar un PID único.
- Asignar memoria para Stack, Text y Datos.
- Crear estructuras de datos asociadas.
- En fork, copiar contexto y regiones correspondientes.

Relación padre-hijo

Respecto de la ejecución, el padre puede continuar concurrentemente con el hijo o puede esperar a que el hijo termine. Respecto del espacio de direcciones, en UNIX el hijo comienza como duplicado del padre; en Windows se crea el proceso y se carga el programa.

11. fork, execve, exit y wait

System Call	Función
fork()	Crea un nuevo proceso igual al llamador. Retorna 0 en el hijo, un valor mayor que 0 en el padre (PID del hijo) y un valor negativo ante error.
execve()	Generalmente se usa después de fork. Carga un nuevo programa en el espacio de direcciones del proceso.
exit()	Termina el proceso y retorna el control al Sistema Operativo.
wait()	Permite al proceso padre esperar la terminación de sus hijos y recibir un código de retorno.
kill	El material menciona que el padre puede terminar la ejecución de sus hijos.
CreateProcess()	En Windows crea un nuevo proceso y carga el programa para ejecución mediante una única System Call.

Regla mental: fork = crea; execve = carga/reemplaza programa; exit = termina; wait = el padre espera.

TEMA 3 - ADMINISTRACIÓN DE MEMORIA

1. MMU

La MMU (Memory Management Unit) es un dispositivo de hardware que mapea direcciones virtuales a direcciones físicas. Es parte del procesador. Reprogramarla es una operación privilegiada y sólo puede realizarse en Kernel Mode.

En el esquema mostrado por el PDF, el valor del registro de realocación se suma a cada dirección generada por el proceso de usuario. Los procesos nunca utilizan directamente direcciones físicas.

2. Espacio de direcciones

Es el rango de direcciones que un proceso puede utilizar para direccionar sus instrucciones y datos. Su tamaño depende de la arquitectura y es independiente de la ubicación real del proceso en RAM.

Arquitectura	Rango conceptual
32 bits	$0 \dots 2^{32} - 1$
64 bits	$0 \dots 2^{64} - 1$

3. Dirección lógica/virtual y dirección física

Dirección	Definición
Lógica / virtual	Referencia una localidad de memoria independientemente de dónde esté realmente ubicada en RAM. Es una dirección dentro del espacio de direcciones del proceso.
Física	Referencia una localidad real de la memoria física/RAM. Es una dirección absoluta.

CPU genera dirección lógica -> traducción/MMU -> dirección física -> RAM

Cuando la resolución se hace en tiempo de ejecución, las direcciones lógicas y físicas son diferentes. Las direcciones lógicas reciben también el nombre de direcciones virtuales.

4. Requisitos de la administración de memoria

El material presenta requisitos como reubicación, protección y compartición.

- Reubicación: el programador no debe necesitar conocer dónde será colocado el proceso en RAM. Un proceso puede salir y volver por swap, incluso en otra ubicación; las referencias deben traducirse según su ubicación actual.
- Protección: los procesos no deben acceder a direcciones de otros procesos salvo que tengan permiso. El chequeo debe realizarse durante la ejecución.
- Compartición: el esquema debe permitir compartir memoria cuando corresponda.

5. Particiones fijas

Es el primer esquema implementado. La memoria se divide en particiones o regiones de tamaño fijo; pueden ser todas del mismo tamaño o no. Cada partición aloja un proceso.

El material enumera como criterios de ubicación: First Fit, Best Fit, Worst Fit y Next Fit.

Fragmentación interna

Se produce en el esquema de particiones fijas. Es la porción de una partición que queda sin utilizar porque el proceso no ocupa todo el espacio asignado.

Particiones fijas -> fragmentación interna.

6. Particiones dinámicas

Son una evolución del esquema anterior. Las particiones varían en tamaño y número; cada una aloja un proceso y se genera dinámicamente con el tamaño justo que necesita.

Fragmentación externa

A medida que los procesos terminan quedan huecos libres separados. Puede haber memoria libre suficiente en total pero no un bloque contiguo suficientemente grande para alojar un proceso. El material menciona la compactación como solución, aunque es muy costosa.

Particiones dinámicas -> fragmentación externa.

Problemas del esquema Base + Límite

- Necesidad de almacenar el espacio de direcciones de forma continua en memoria física.
- Fragmentación.
- Mantener partes del proceso que no son necesarias.
- Los esquemas de particiones fijas y dinámicas no se usan hoy en día según el material.
- Las soluciones presentadas luego son Segmentación y Paginación.

7. Segmentación

La segmentación se asemeja a la visión lógica del usuario: el programa se divide en partes o secciones. Un segmento es una unidad lógica, por ejemplo programa principal, procedimientos y funciones, variables locales/globales o stack.

Dirección lógica y estructuras

Una dirección lógica se expresa como (segmento, desplazamiento). Existe una Tabla de Segmentos. Cada entrada contiene una Base y un Límite.

Elemento	Función
Base	Dirección física donde comienza el segmento.
Limit	Longitud/límite del segmento.
STBR	Apunta a la tabla de segmentos.
STLR	Indica la cantidad de segmentos del programa.

Para resolver una dirección se identifica el segmento, se verifica que el desplazamiento esté dentro del límite y se utiliza la base para obtener la dirección física. La segmentación puede producir fragmentación externa.

8. Paginación

La memoria física se divide en bloques del mismo tamaño llamados Marcos (Frames). El espacio de direcciones lógico se divide en bloques del mismo tamaño llamados Páginas (Pages).

Tamaño de página = tamaño de marco.

El SO mantiene una tabla de páginas por proceso. Cada entrada indica, entre otras cosas, en qué marco se encuentra una página.

Dirección lógica = número de página * desplazamiento

Traducción: Página -> consultar tabla de páginas -> obtener marco -> conservar el desplazamiento -> dirección física.

La paginación es transparente al programador y elimina la fragmentación externa. El material no dice que elimine toda fragmentación: puede quedar espacio sin utilizar dentro de la última página asignada, es decir, fragmentación interna.

9. Segmentación paginada

Combina segmentación y paginación. La segmentación aporta visión lógica, modularidad y soporte para compartir/proteger; la paginación es transparente al programador y elimina la fragmentación externa.

En segmentación paginada, cada segmento se divide en páginas de tamaño fijo. La dirección lógica se interpreta conceptualmente como:

Segmento | Página | Desplazamiento

La traducción se realiza en dos etapas: primero se utiliza la información del segmento y luego la tabla de páginas correspondiente.

10. Huecos explícitos entre la guía y los PDFs

La guía pide algunos análisis que los PDFs aportados no desarrollan con suficiente detalle. Para respetar el criterio de estudiar sólo desde las fuentes entregadas, estos puntos quedan marcados y no se completan con teoría externa.

Punto pedido por la guía	Qué soportan los PDFs aportados
Particiones fijas de igual vs. diferente tamaño: ventajas y desventajas	El PDF establece que las particiones fijas pueden ser todas del mismo tamaño o no, pero no desarrolla una comparación explícita de ventajas/desventajas.
First Fit, Best Fit, Worst Fit y Next Fit: funcionamiento y ventajas/desventajas individuales	El PDF los enumera como criterios de ubicación, pero no desarrolla individualmente su funcionamiento ni una tabla de ventajas/desventajas.

Lo que sí puede afirmarse con las fuentes: en particiones fijas una partición aloja un proceso y puede haber fragmentación interna; en particiones dinámicas las particiones cambian de tamaño/número y aparece fragmentación externa.

REPASO FINAL - MAPA MENTAL

La estructura completa del primer teórico puede recordarse así:

INTRODUCCIÓN

SO -> objetivos -> apoyo del hardware -> Kernel/Shell/Herramientas -> modo Usuario/Kernel -> interrupciones/traps -> System Calls

CPU Y PROCESOS

Programa -> Proceso -> PCB -> Scheduling -> Apropiativo/No apropiativo -> Schedulers -> Loader/Dispatcher -> Context Switch -> CPU/I/O Bound -> Estados -> Creación -> fork/execve/wait/exit

MEMORIA

Espacio de direcciones -> lógica/virtual vs. física -> MMU -> Base/Límite -> Particiones fijas -> Fragmentación interna -> Particiones dinámicas -> Fragmentación externa -> Segmentación -> Paginación -> Segmentación paginada

Conexión central entre los temas

Un proceso ejecutándose en modo usuario no puede controlar directamente CPU, memoria ni E/S. Cuando necesita un servicio protegido realiza una System Call; se produce un trap/excepción, la CPU pasa a modo Kernel, el kernel ejecuta el servicio y luego se retorna al proceso.

Checklist de la guía

- Definir Sistema Operativo y sus objetivos.
- Explicar el apoyo de hardware: modos, clock y protección de memoria/E/S/CPU.
- Distinguir Kernel, Shell/interfaz y herramientas.
- Explicar System Calls, categorías y diferencias x86 32/64 del anexo.
- Distinguir programa y proceso; conocer Text, Datos y Stack.
- Explicar PCB y diferenciarla del espacio de direcciones.
- Explicar metas de Scheduling, Batch e Interactivos.
- Distinguir apropiativo y no apropiativo.
- Explicar Long/Short/Medium Term Scheduler, Loader y Dispatcher.
- Explicar Context Switch y su costo.
- Distinguir CPU-bound e I/O-bound.
- Explicar estados y transiciones de procesos.
- Explicar creación/terminación y fork, execve, wait, exit.
- Definir MMU, espacio de direcciones, dirección lógica/virtual y física.
- Explicar particiones fijas/dinámicas y sus fragmentaciones.
- Explicar segmentación y sus estructuras.
- Explicar paginación, páginas, marcos y tabla de páginas.
- Explicar segmentación paginada.
- Recordar los dos huecos de la guía no desarrollados por los PDFs.

Material de estudio preparado a partir de la guía y los PDFs aportados para el primer parcial. No se incorporó teoría externa para completar puntos no desarrollados por las fuentes.